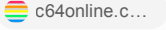


Sí. Y, de hecho, **podemos plantearlo como un pequeño “motor de plataformas” para C64**, de manera que después puedas cambiar personajes, enemigos, niveles, gravedad, velocidad, etc., sin tener que reescribir todo el juego.

C64 Online Emulator que estás usando permite cargar programas `.PRG` directamente y también tiene joystick simulado; el joystick 1 está mapeado a **A/D/W/S + Shift** y el joystick 2 a las teclas de cursor. 

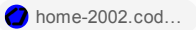
Lo primero: BASIC V2 o ensamblador

Para lo que describes:

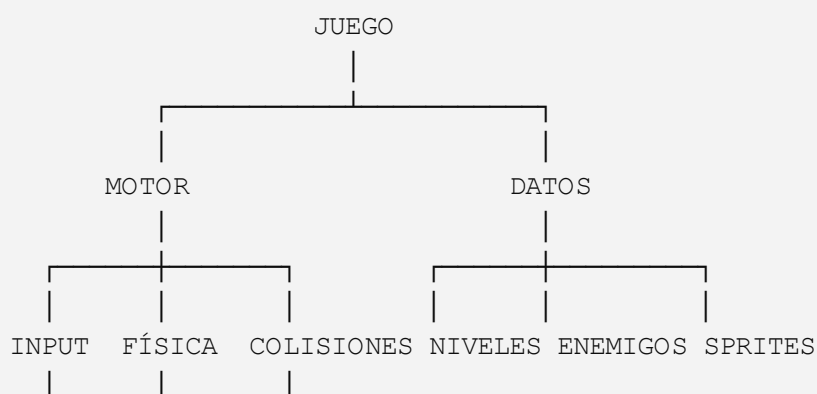
- personaje con **gravedad**
- salto
- plataformas
- enemigos
- colisiones
- desplazamiento horizontal
- varias pantallas
- sprites animados
- cámara que siga al jugador
- posibilidad de editar niveles

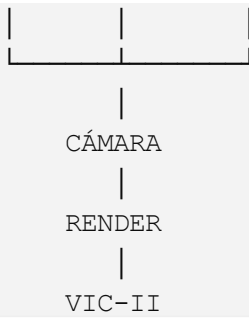
no te recomiendo hacerlo todo en BASIC V2 puro.

El BASIC V2 es estupendo para aprender y para prototipos, pero el scroll y la actualización continua de pantalla son demasiado lentos. De hecho, el scroll horizontal suave del C64 se realiza utilizando los registros del VIC-II y normalmente requiere código máquina; hacerlo exclusivamente desde BASIC resulta muy lento. 

El C64, además, tiene **8 sprites hardware**, cada uno de 24×21 píxeles, que son precisamente una de las piezas fundamentales para este tipo de juego. 

Yo haría esta arquitectura





Y separaríamos **motor** y **datos**.

Por ejemplo:

```
PLAYER
  X
  Y
  VX
  VY
  GRAVITY
  JUMP
  SPEED
  SPRITE
  FRAME
```

```
ENEMY
  TYPE
  X
  Y
  VX
  VY
  SPRITE
  LIFE
  AI
```

```
LEVEL
  WIDTH
  HEIGHT
  DATA[]
  ENEMIES[]
  START_X
  START_Y
```

Así podrías tener:

```
ENEMIGO 0 = murciélago
ENEMIGO 1 = gusano
ENEMIGO 2 = soldado
ENEMIGO 3 = proyectil
```

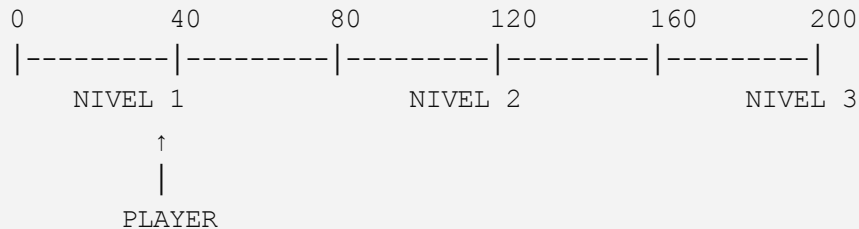
y posteriormente cambiar únicamente los datos.

Una idea importante para el scroll

No moveríamos realmente todo el escenario píxel por píxel.

Tendríamos un **mundo mucho más grande que la pantalla**:

MUNDO



La pantalla sería solamente una **ventana** sobre ese mundo.

Por ejemplo:

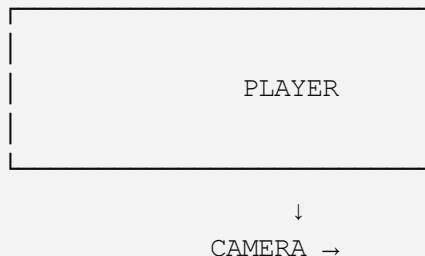
```
PLAYER_X = 1350
CAMERA_X = 1200
SCREEN_X = PLAYER_X - CAMERA_X
```

De esta forma:

mundo:	1200	1201	1202	1203	1204...
	↓				
pantalla:	0	1	2	3	4...

Y cuando el jugador llegue aproximadamente al 60–70 % de la pantalla:

PLAYER → → → → →



la cámara empieza a desplazarse.

Física

La física puede ser muy sencilla.

Cada frame:

```
VX = velocidad horizontal  
VY = velocidad vertical
```

```
VY = VY + GRAVITY
```

```
X = X + VX
```

```
Y = Y + VY
```

Por ejemplo:

```
GRAVITY = 1
```

```
JUMP = -8
```

```
SPEED = 3
```

Al pulsar salto:

```
VY = -8
```

y después:

```
-8  
-7  
-6  
-5  
-4  
-3  
-2  
-1  
0      ← punto máximo  
1  
2  
3  
4  
...
```

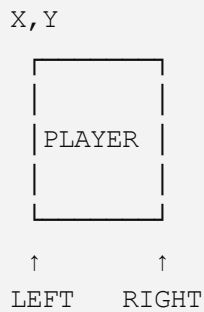
Hasta que choque con el suelo.

Eso nos permite conseguir una sensación de plataformas bastante convincente.

Colisiones

Aquí es donde merece la pena diseñarlo bien desde el principio.

Podemos hacer que el personaje tenga una caja:



Y cada plataforma:



Comprobamos:

```
PLAYER_BOTTOM >= PLATFORM_TOP
```

y si además existe solapamiento horizontal:

```
PLAYER_RIGHT > PLATFORM_LEFT  
PLAYER_LEFT < PLATFORM_RIGHT
```

entonces:

```
PLAYER_Y = PLATFORM_TOP - PLAYER_HEIGHT  
VY = 0  
ON_GROUND = 1
```

Con eso ya tenemos:

- gravedad
- suelo
- plataformas
- salto
- caída
- aterrizaje.

Y podemos hacer niveles editables

Esta es la parte que creo que te va a interesar más.

En lugar de programar:

basic

```
PRINT "PLATAFORMA"
```

podemos representar el nivel mediante datos.

Por ejemplo:

```
.....  
.....  
.....###.....  
.....  
.....####.....#####.....  
.....  
#####.....#####.....
```

Donde:

```
. = vacío  
# = plataforma  
P = jugador  
E = enemigo  
C = moneda  
G = puerta
```

Por ejemplo:

```
.....  
.....  
.....E.....E.....  
.....###.....  
.....###.....  
.....  
.P.....###.....  
#####
```

Esto es muy interesante porque posteriormente podemos crear un **editor de niveles** en el propio C64 o incluso generar los datos desde el PC.

Sprites

Los sprites del C64 son ideales para:

```
SPRITE 0 → jugador  
SPRITE 1 → enemigo 1  
SPRITE 2 → enemigo 2  
SPRITE 3 → enemigo 3
```

```
SPRITE 4 → objeto  
SPRITE 5 → proyectil  
...
```

Pero hay una limitación importante: **solo existen 8 sprites hardware simultáneos**.

Por eso, si queremos hacer un juego más ambicioso, podemos implementar posteriormente un **sprite multiplexer**, que reutiliza los 8 sprites hardware para mostrar más objetos. Hay juegos modernos de C64 que utilizan esta técnica para mostrar bastantes más elementos en pantalla.  Red...

No empezaría por ahí. Primero haríamos que el motor funcione con 3–4 objetos.

Te propongo hacerlo por fases

FASE 1 — Motor mínimo

[Jugador]

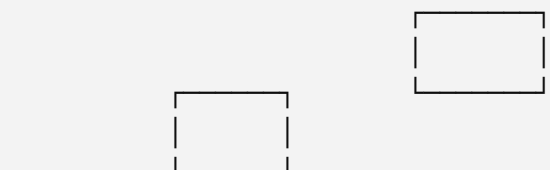
↓ gravedad

suelo

Implementamos:

- sprite del jugador
- movimiento izquierda/derecha
- gravedad
- salto
- suelo
- colisión

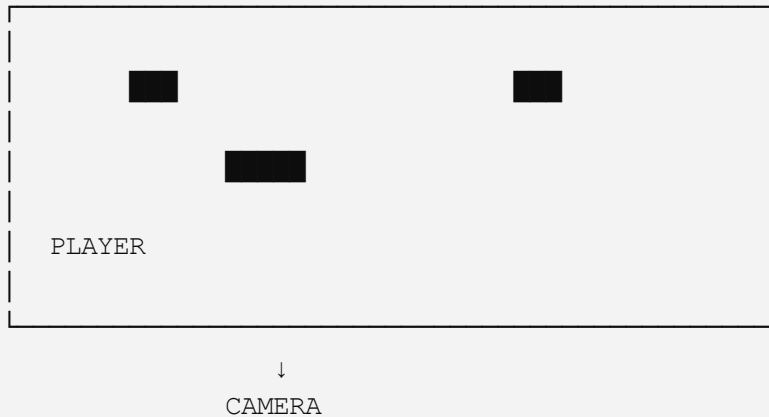
FASE 2 — Plataformas



Añadimos:

- múltiples plataformas
 - caída entre plataformas
 - salto
 - colisiones laterales
 - techo
-

FASE 3 — Mundo



Añadimos:

- coordenadas de mundo
 - cámara
 - scroll horizontal
 - niveles largos
-

FASE 4 — Enemigos

Por ejemplo:

```
TYPE 0
Patrulla izquierda/derecha

TYPE 1
Persigue al jugador

TYPE 2
Salta

TYPE 3
Dispara

TYPE 4
```

Cada enemigo tendrá:

```
X
Y
VX
VY
TYPE
STATE
SPRITE
LIFE
```

FASE 5 — Sistema de niveles

Entonces podremos definir:

```
LEVEL 1
  ancho = 200
  spawn = 10,18
  enemigos = ...
  plataformas = ...

LEVEL 2
  ancho = 350
  spawn = 5,18
  enemigos = ...

LEVEL 3
  ...
```

FASE 6 — Sprites personalizables

Y aquí podríamos tener algo realmente interesante:

```
SPRITE PLAYER
FRAME 0 → parado
FRAME 1 → caminando
FRAME 2 → caminando
FRAME 3 → saltando
```

y:

```
ENEMY 1
FRAME 0
```

```
FRAME 1
FRAME 2
FRAME 3
```

El motor simplemente sabe:

```
SPRITE = PLAYER_SPRITE
FRAME = PLAYER_FRAME
```

No necesita saber cómo está dibujado.

¿Y el código?

Yo **no empezaría escribiendo directamente el juego completo en BASIC**.

Haría primero un **prototipo en BASIC V2 para comprobar la lógica**, y después pasaríamos las partes críticas a **6502 Assembly**.

La estructura final podría ser aproximadamente:

```
C64 PLATFORM ENGINE
|
|— INIT
|
|— INPUT
|
|— PLAYER
|   |— movement
|   |— gravity
|   |— jump
|   └─ collision
|
|— ENEMIES
|   |— enemy_type_0
|   |— enemy_type_1
|   └─ enemy_type_2
|
|— LEVEL
|   |— load
|   |— collision_map
|   └─ objects
|
|— CAMERA
|   └─ horizontal_scroll
|
|— SPRITES
|   |— player
|   |— enemies
|   └─ animation
```

```
|
└─ GAME
    ├── start
    ├── play
    ├── death
    └─ next_level
```

Y podemos hacerlo contigo paso a paso dentro de ese emulador, empezando por algo muy pequeño: un cuadrado/sprite que se mueve, cae por gravedad y salta sobre una plataforma. Después añadimos scroll y vamos convirtiéndolo en un verdadero motor de plataformas.

Además, podemos intentar que el código sea **reutilizable**, de forma que al final puedas cambiar algo como:

```
PLAYER_SPRITE = 10
ENEMY_1_SPRITE = 11
ENEMY_2_SPRITE = 12

GRAVITY = 1
JUMP_FORCE = -9
PLAYER_SPEED = 3

LEVEL = 1
```

sin tocar el motor.

Si quieres hacerlo en el emulador C64online que has enlazado, empezaría por la FASE 1 y te puedo dar el primer programa BASIC V2 completo para copiar/pegar en el C64, y después iremos sustituyendo progresivamente BASIC por 6502 donde haga falta para conseguir el scroll fluido.